

Practical – 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

A. Theory

1. How is JSON fetched, parsed, and rendered?

JSON data is fetched using the **Fetch API** with `fetch()`. The response is converted into JavaScript objects using `response.json()`. The data is then processed and dynamically rendered as HTML cards inside the `dataContainer` element using JavaScript.

2. Which array methods are used for search, filter, sort, and pagination?

- **Search:** `filter()` is used to find records matching the search text.
- **Filter:** `filter()` is used to filter events by category.
- **Sort:** `sort()` is used to arrange records in ascending or descending order.
- **Pagination:** `slice()` is used to select only the records belonging to the current page.

3. How are loading and error states handled?

A **loading message** is displayed while the JSON file is being fetched. If the fetch operation fails, an **error message** is shown to the user. The implementation also provides fallback data when the page is opened directly without a local server, where browser Fetch restrictions may occur.

4. How is modularity maintained in JavaScript files?

JavaScript functionality is organized into separate logical sections. The existing **Practical 4/5 functionality** is preserved, while Practical 6 functionality is grouped into its own section. Common functions handle fetching, rendering, searching, filtering, sorting, and pagination, allowing the same JavaScript file to be reused by `events.html` without duplicating code.

B. Implementation/ Output/Result:

In Practical 6, the existing **StudentHubPortal** project was extended to implement dynamic rendering of external JSON data using the **Fetch API**, along with search, filtering, sorting, and pagination. A new `events.html` page was added as the main interface for this practical. The page uses the existing StudentHub Portal header, navigation, logo, `style.css`, and `script.js`, so the new functionality remains consistent with the rest of the project.

Three external JSON datasets were created: `events.json`, `students.json`, and `faqs.json`. Each dataset contains **15 records**. The `events.json` file contains event details such as event title, date, category, and location. The `students.json` file contains student name, email, course, and year, while `faqs.json` contains questions and answers. The `events.html` page provides a data selection option for Events, Students, and FAQs.

The JSON data is loaded dynamically through the **Fetch API** using the path `../json/` followed by the selected JSON filename. The response is checked using `response.ok` and converted into JavaScript objects using `response.json()`. After loading the data, the records are stored in the `records` array and dynamically displayed inside the `dataContainer` section. Separate card layouts are generated for events, students, and FAQs using the `makeCard()` function.

A **search feature** is implemented using the `filter()` array method. The entered search text is compared with the complete JSON record using `JSON.stringify()` and `toLowerCase()`, allowing the user to search through the available information. For events, a category filter is generated dynamically from the categories present in the JSON data. For student records, the filter is generated from the available courses.

The implementation also provides **sorting** through a dropdown containing Default, A-Z, and Z-A options. The `sort()` method with `localeCompare()` is used to arrange event titles, student names, or FAQ questions alphabetically. **Pagination** is implemented with four records per page. The `slice()` method selects the records for the current page, while Previous and Next buttons allow the user to navigate between pages.

Loading and error handling are also included. While data is being requested, the status displays "**Loading JSON data...**". If the JSON request fails, the project uses the locally defined fallback dataset and displays a message explaining that local data is being used because direct file access can block Fetch. This allows the practical to be tested without depending on a local server.

The Practical 6 JavaScript functionality is placed in a dedicated `/* ===== PRACTICAL 6 ===== */` section of the existing `script.js` file. An immediately invoked function is used to keep the practical's variables and functions organized. Functions such as `loadData()`, `setFilter()`, `showData()`, and `makeCard()` divide the functionality into

separate tasks. The existing Practical 4 and Practical 5 JavaScript functionality is preserved, while Practical 6 reuses the common CSS and JavaScript structure.

Therefore, Practical 6 successfully adds **external JSON data loading, dynamic rendering, search, category/course filtering, A-Z/Z-A sorting, and four-record pagination** to the StudentHubPortal while maintaining the existing project's design and functionality.

C. Evidence of Work (Screenshots):

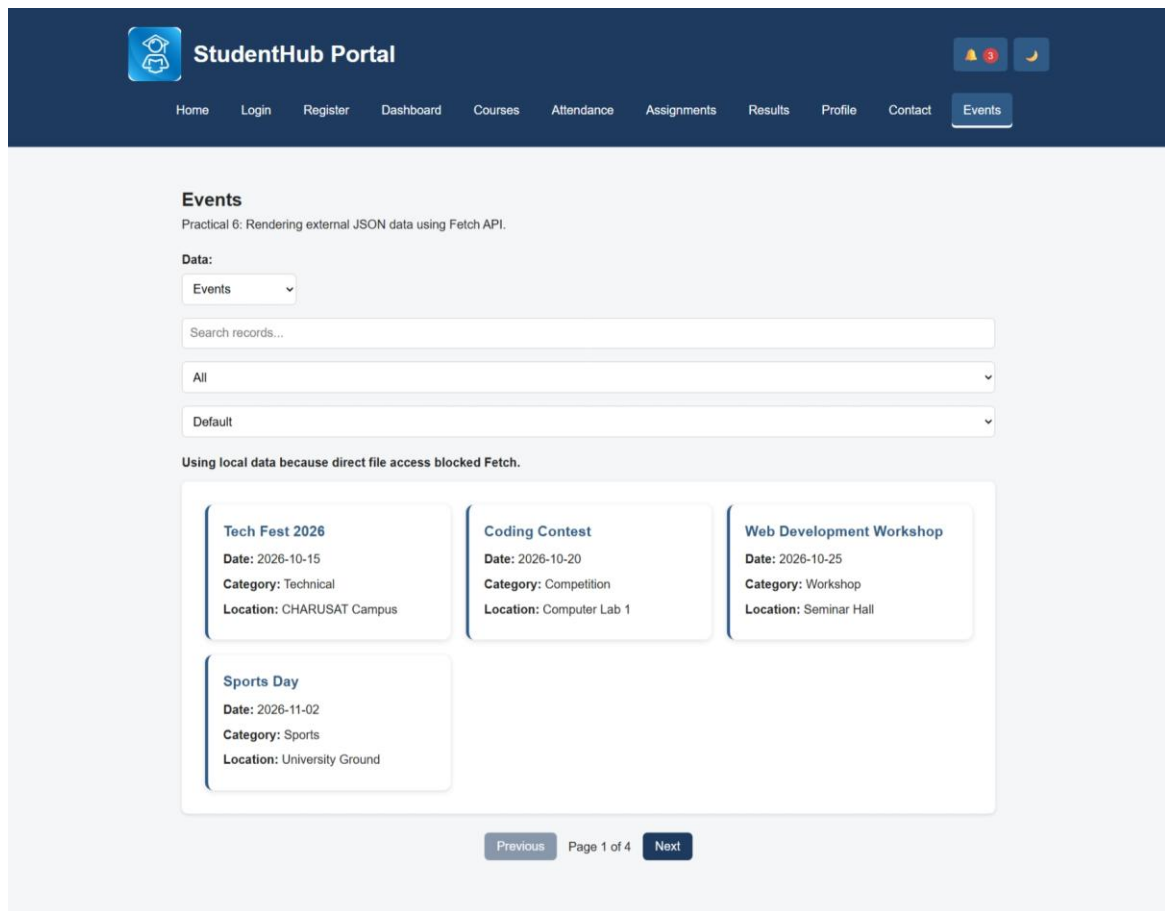


Fig1: Events Page – Initial View

The screenshot shows the 'StudentHub Portal' interface. The header includes a logo, navigation links (Home, Login, Register, Dashboard, Courses, Attendance, Assignments, Results, Profile, Contact, Events), and a notification bell with 3 alerts. The 'Events' section is active, displaying 'Practical 6: Rendering external JSON data using Fetch API.' Below this, there's a 'Data:' dropdown set to 'Students', a search bar, and filters for 'All' and 'Default'. A message states 'Using local data because direct file access blocked Fetch.' The main content area displays four student profiles in a grid:

- Aarav Patel**
Email: aarav.patel@example.com
Course: BTech IT
Year: 2
- Riya Shah**
Email: riya.shah@example.com
Course: BTech CSE
Year: 2
- Dev Mehta**
Email: dev.mehta@example.com
Course: BTech IT
Year: 3
- Neha Joshi**
Email: neha.joshi@example.com
Course: BTech CE
Year: 2

At the bottom, there are 'Previous', 'Page 1 of 4', and 'Next' navigation buttons.

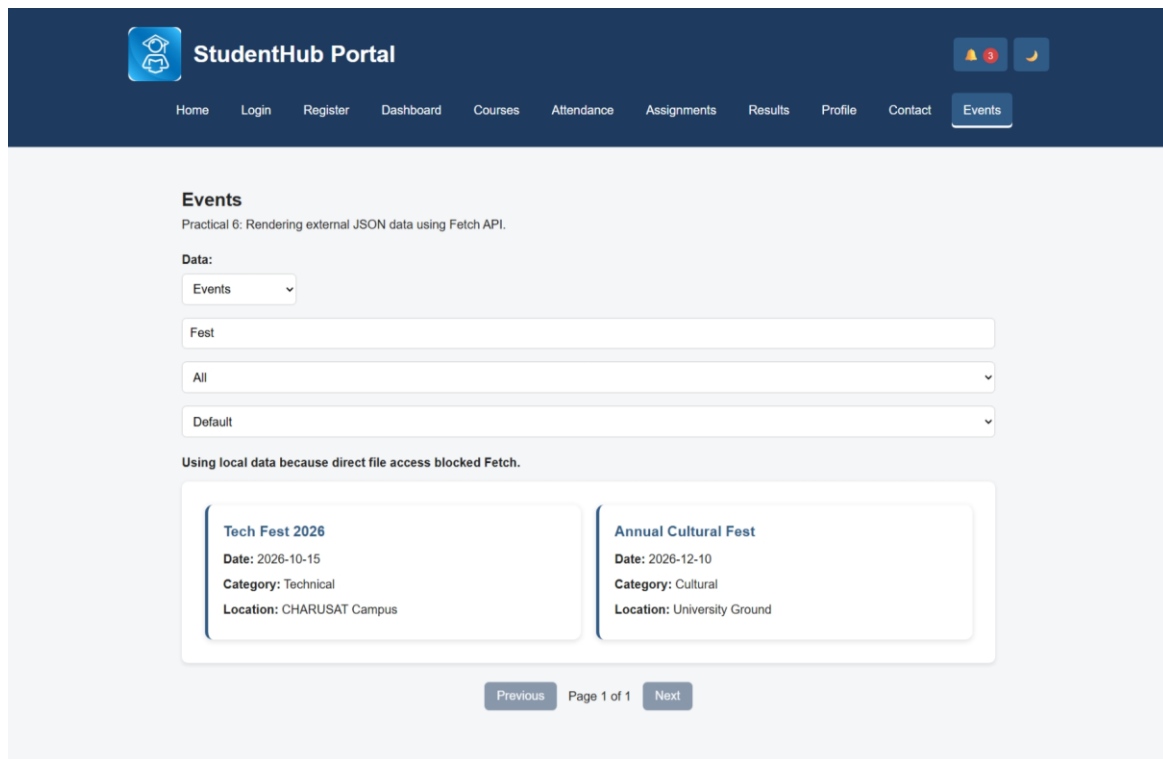
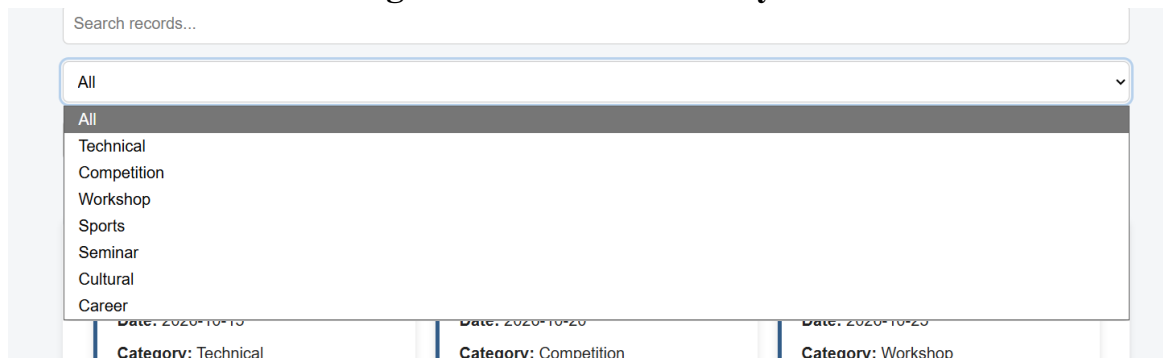
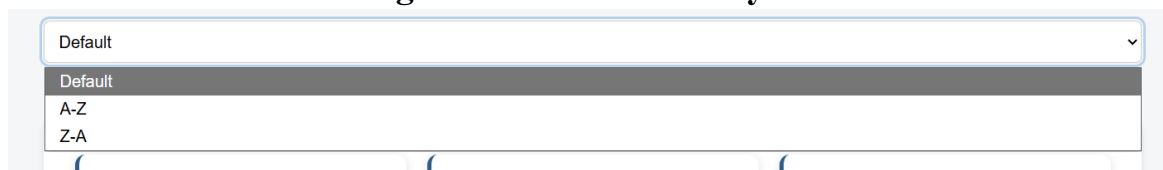
Fig2: JSON Data / Dynamic Rendering-Students

The screenshot shows the 'StudentHub Portal' interface with the 'Events' section active. The 'Data:' dropdown is set to 'FAQs'. The search bar and filters remain the same. The message 'Using local data because direct file access blocked Fetch.' is present. The main content area displays four FAQ items in a grid:

- What is StudentHub?**
StudentHub is a portal for managing student information and activities.
- How do I register?**
Open the Register page and fill in the required details.
- How do I login?**
Use your registered email and password on the Login page.
- Where can I see events?**
Events can be viewed in the StudentHub event section.

At the bottom, there are 'Previous', 'Page 1 of 4', and 'Next' navigation buttons.

Fig3: JSON Data / Dynamic Rendering-FAQs

**Fig4: Search Functionality****Fig5: Filter Functionality****Fig6: Sorting Functionality**

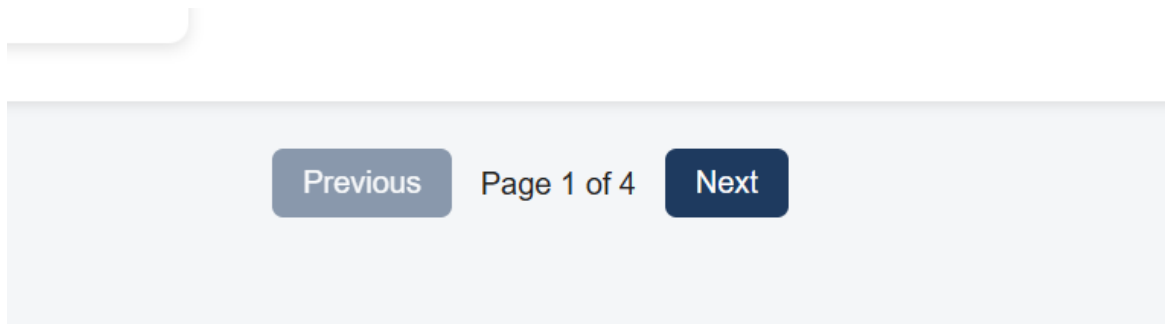


Fig7: Pagination – First Page

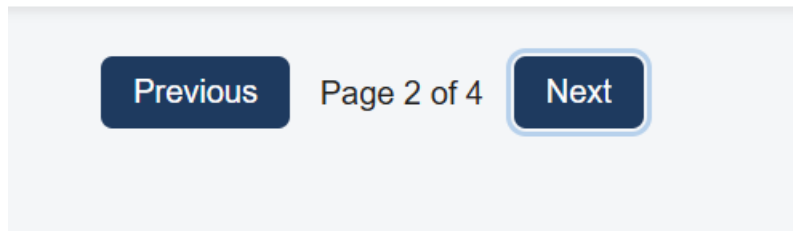


Fig8: Pagination – Another Page

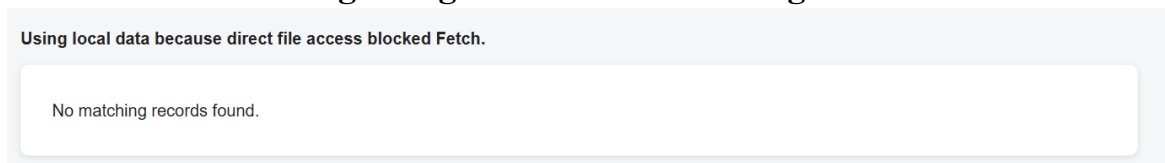


Fig9: Loading / Error Handling

D. GitHub Repository Link:

[aryan150806-wq/StudentHubPortal: Web Development Framework P](https://github.com/aryan150806-wq/StudentHubPortal)

E. Challenges Faced:

1. Fetching and Parsing JSON

Learned to retrieve external JSON data using the Fetch API and convert it into JavaScript objects using `response.json()`.

2. Dynamic Data Rendering

Learned to take JSON records and dynamically create and display HTML cards using JavaScript and DOM manipulation.

3. **Search, Filter, and Sort**

Implemented search and filtering using `filter()` and alphabetical ordering using the `sort()` method.

4. **Pagination**

Learned to divide the records into pages using `slice()` and provide Previous/Next navigation for the displayed data.

5. **Error Handling and Modularity**

Learned to handle loading and fetch errors while keeping Practical 6 functionality organized within the existing JavaScript structure.

F. Learning Outcomes:

This practical helped in understanding how to use the **Fetch API** to retrieve and parse external JSON data and dynamically display it on a web page. It also developed the ability to implement **search, filtering, sorting, and pagination** using JavaScript array methods. Additionally, it improved understanding of **loading and error handling** and maintaining modular JavaScript functionality while integrating the new features into the existing StudentHubPortal project.

Faculty Remarks: